Vidya Mitra

Feature & Architecture Roadmap

Current Build Plan — Backend & LLM Workflows
Future Upgrades — Prioritized Roadmap

Prepared by: Software Architecture & Engineering Lead
June 27, 2026

Executive Summary

This document is the working build plan for Vidya Mitra. Section 1 covers every feature we've scoped so far, each broken into the exact path a request takes through the mobile client, the Spring Boot backend, and the AI/LLM layer — so any engineer picking up a feature knows precisely which services it touches. Section 2 is a forward-looking set of upgrades I'd recommend we evaluate next, organized by how much effort each takes versus how directly it improves a student's actual learning outcome, not just engagement metrics.

1. Current Features — Build Plan

Everything below has been scoped in our architecture discussions. Each feature lists the backend services, data stores, and LLM components it depends on.

1. Fine-Tuned Multilingual Tutor LLM
2. RAG-Grounded Topic Explanation
3. Live Camera Translation (Parent Feature)
4. Voice Tutor Interface
5. Auto-Generated Tests & Exercises
6. Smart Test Scheduling
7. Parent Progress Dashboard
8. Trust & Source Citation Layer
9. Socratic Strictness Dial
10. Parent Mode (Pedagogical Concept Translation)
11. Study Together Mode
12. AI-Generated Lesson Diagrams
13. Offline-First / Low-Bandwidth Mode

1. Fine-Tuned Multilingual Tutor LLM

CORE AI

Phi-3-mini-4k-instruct fine-tuned with QLoRA on NCERT-grounded Q&A; pairs in 5 Indian languages (Hindi, English, Tamil, Telugu, Bengali to start), giving the tutor a consistent, simple, encouraging voice across every subject.

#	COMPONENT	WHAT HAPPENS
1	**Dataset prep (offline)**	prepare_dataset.py calls Claude to turn raw NCERT chapter text into child-friendly Q&A; pairs, grounded only in that chapter's content, tagged by language/subject/class.
2	**QLoRA fine-tuning**	train_phi3_qlora.py fine-tunes Phi-3-mini in 4-bit on the combined multilingual dataset using PEFT + TRL's SFTTrainer.
3	**Model export**	merge_and_export.py merges the LoRA adapter into the base model; the result is converted to GGUF and quantized for local serving.
4	**Runtime serving**	Ollama hosts the merged model locally; Spring Boot's Tutor Service calls it for every tutoring request.

2. RAG-Grounded Topic Explanation

RAG / RETRIEVAL

Every tutor answer is grounded in the actual NCERT textbook rather than the model's general knowledge, to keep explanations accurate for young children.

#	COMPONENT	WHAT HAPPENS
1	**Spring Boot endpoint**	/api/tutor/ask receives the student's question with class, subject, and language context.
2	**Metadata filter**	ChromaDB query is pre-filtered to chunks matching the student's exact class level and subject before any semantic search runs.
3	**Hybrid search + rerank**	Dense embeddings and BM25 keyword search run in parallel; a cross-encoder reranks the combined candidates for precision.
4	**Groundedness check**	The agent verifies the retrieved chunks actually support an answer; if not, it returns a safe fallback instead of guessing.
5	**Generation**	The fine-tuned Phi-3-mini model generates the answer, citing the supporting NCERT chunk IDs, logged to MongoDB.

3. Live Camera Translation (Parent Feature)

MOBILE CLIENT

A parent points the camera at the English-medium textbook; the sentence is translated into their language directly on the camera view, in real time.

#	COMPONENT	WHAT HAPPENS
1	**React Native client**	Vision Camera captures frames; ML Kit's on-device Latin-script OCR detects English text and debounces for 500ms to avoid flicker.
2	**Translation**	Detected text is sent to the on-device IndicTrans2-Distilled model (English-to-Indic), quantized to ~60-90MB for offline use.
3	**AR overlay**	Translated text is rendered back onto the live camera frame in a canvas layer.
4	**Logging**	The scan is logged as a session_type=camera_translate document in MongoDB for the parent's usage history.

4. Voice Tutor Interface

VOICE

Children and parents can speak their question instead of typing, and receive a spoken answer back, in their preferred language.

#	COMPONENT	WHAT HAPPENS
1	**Speech-to-text**	Whisper-small (or Gemini Live API for full two-way sessions) transcribes the spoken question.
2	**Tutor pipeline**	The transcribed text is routed through the same RAG-grounded explanation pipeline as a typed question.
3	**Text-to-speech**	The generated answer is converted to audio via a TTS engine (Coqui/Parler-TTS, or Gemini TTS), tuned for a warm, slow, clear delivery.
4	**Message logging**	Both the transcript and the audio_url are stored on the message document in MongoDB.

5. Auto-Generated Tests & Exercises

BACKEND

Quizzes are generated automatically from the topics a student has actually studied, rather than pulled from a static, hand-written bank.

#	COMPONENT	WHAT HAPPENS
1	**Trigger**	Either an on-demand call to /api/tests/generate/{studentId} or the weekly Spring Boot scheduler.
2	**RAG retrieval**	The relevant NCERT chunks for the topic are retrieved to ground the generated questions.
3	**Structured generation**	LangChain's structured-output chain prompts Phi-3-mini to return JSON: question, options, correct_answer, explanation, difficulty.
4	**Storage**	Generated questions are written to the PostgreSQL questions table, tagged by subject, class, chapter, and difficulty.

6. Smart Test Scheduling

BACKEND

Tests are scheduled automatically based on what a student studied that week, plus a couple of review questions from past weak areas.

#	COMPONENT	WHAT HAPPENS
1	**Weekly scheduler**	A Spring Boot cron job runs once a week per active student.
2	**Context read**	Reads current_topics from the student's MongoDB profile and recent score history from PostgreSQL.
3	**Selection**	Picks 5 new questions from this week's topics and 2 review questions from previously weak areas.
4	**Notification**	A scheduled test record is created in PostgreSQL and a Firebase Cloud Messaging push notification is sent to the parent's device.

7. Parent Progress Dashboard

BACKEND

A weekly summary of what the child studied, how they performed, and where they need more practice, written in plain language the parent can understand.

#	COMPONENT	WHAT HAPPENS
1	**Weekly aggregation**	A Sunday-night Spring Boot job rolls up the week's sessions and test scores per student.
2	**AI summarization**	The fine-tuned model generates a short parent_note narrative summarizing the week, in the parent's preferred language.
3	**Storage**	The rollup is written to the progress collection in MongoDB, keyed by student_id and ISO week.
4	**Display**	The React Native parent dashboard reads it via /api/progress/{studentId}/weekly and renders charts plus the narrative note.

8. Trust & Source Citation Layer

TRUST & SAFETY

Every tutor answer visibly shows which NCERT chapter and page it came from, so parents can verify the app isn't making things up.

#	COMPONENT	WHAT HAPPENS
1	**Source tracking**	The RAG groundedness check (see Topic Explanation above) attaches rag_chunk_ids to every assistant message.
2	**Chunk resolution**	Spring Boot resolves chunk IDs to human-readable references (e.g. 'Class 3 EVS, Ch.4, p.12') via the ChromaDB metadata.
3	**UI rendering**	The React Native chat bubble displays a small 'Source:' tag beneath the answer.

9. Socratic Strictness Dial

PERSONALIZATION

Parents control how many hints the tutor gives before just explaining the answer outright, instead of one fixed behavior for every family.

#	COMPONENT	WHAT HAPPENS
1	**Setting storage**	A strictness value (1-5) is stored in learning_preferences on the student's MongoDB profile.
2	**Prompt injection**	The LangChain agent reads this value and inserts a matching instruction into the system prompt ('give at most one hint before explaining' vs. 'always explain fully').
3	**Effect**	The fine-tuned model's response behavior shifts accordingly for that session, without any retraining needed.

10. Parent Mode (Pedagogical Concept Translation)

PERSONALIZATION

Goes beyond translating words — explains curriculum concepts (like 'EVS' or 'phonics') that the parent never encountered in their own schooling, in their own vocabulary.

#	COMPONENT	WHAT HAPPENS
1	**Trigger**	Parent taps 'explain this to me' on a term inside a translated sentence.
2	**Persona routing**	Spring Boot sends the request to /api/tutor/ask with a persona=parent flag.
3	**Prompt selection**	The LangChain agent swaps in a parent-oriented system prompt template, distinct from the child-facing one, targeting adult comprehension.
4	**Grounded generation**	The same RAG pipeline supplies the underlying NCERT content; the model explains it in the parent's language and frame of reference.

11. Study Together Mode

MOBILE CLIENT

One camera scan or question produces two outputs at once: a simplified explanation for the child and a contextual explanation for the parent, shown side by side.

#	COMPONENT	WHAT HAPPENS
1	**Single trigger**	One scan or question is submitted from the React Native UI.
2	**Parallel chains**	Spring Boot fans the request out to two LangChain chains in parallel: a child-persona chain and a parent-persona chain.
3	**Shared retrieval**	Both chains reuse the same RAG retrieval call to stay consistent with each other.
4	**Split-screen render**	The app displays both outputs together; both are logged under one session with two tagged message threads.

12. AI-Generated Lesson Diagrams

VISUAL AI

Topic explanations can be accompanied by a generated diagram or infographic — useful for visual concepts like the water cycle or plant parts.

#	COMPONENT	WHAT HAPPENS
1	**Trigger**	A topic explanation flagged as visual-friendly requests an illustration.
2	**Image generation**	Spring Boot calls the Gemini API (Nano Banana Pro) with the topic and target language for legible multilingual labels.
3	**Caching**	Generated images are cached (Redis/CDN) so the same topic+language combination isn't regenerated repeatedly.
4	**Human review**	Diagrams are flagged for a human spot-check before being promoted into the reusable diagram library, given that generated diagrams can mislabel a process.

13. Offline-First / Low-Bandwidth Mode

DATA & INFRA

The app stays usable on intermittent rural connectivity by caching common explanations and falling back to an on-device model when offline.

#	COMPONENT	WHAT HAPPENS
1	**Connectivity check**	The React Native client detects connection quality before each request.
2	**Cache lookup**	Common topic explanations are served instantly from the topics_cache MongoDB collection (mirrored to Redis for hot topics).
3	**On-device fallback**	If offline, a small on-device model (via llama.cpp bindings) answers from cached NCERT content directly on the phone.
4	**Queue and sync**	Unresolved requests queue locally and sync with the backend once connectivity returns.

2. Future Upgrades — Roadmap

These aren't in the current build plan yet — they're what I'd put in front of the team next. Each is tagged by impact on the student's actual learning and the engineering effort to build it, and grouped into three tiers so we can sequence sensibly rather than chasing every idea at once.

Tier 1 — Quick Wins (ship soon, low effort)

1. "Explain It Back to Me" — Active Recall Mode

IMPACT: HIGH EFFORT: LOW

After the tutor explains a concept, it asks the child to explain it back in their own words — flipping the interaction instead of staying one-directional.

→ Why it helps the student: Active recall and the 'protege effect' are some of the most evidence-backed ways young children retain concepts — explaining something back cements it far better than re-reading or re-hearing it.

#	COMPONENT	WHAT HAPPENS
1	**Prompt addition**	After any explanation, the LangChain agent appends a short 'now you tell me' follow-up turn.
2	**Evaluation**	The child's spoken or typed response is sent back through the LLM with a rubric prompt: does this capture the core idea, gently, no harsh grading.
3	**Encouragement loop**	The model responds with specific praise plus one gap-filling correction if needed, logged as a distinct message type for the progress dashboard.

2. Spaced-Repetition Micro-Review

IMPACT: HIGH EFFORT: LOW-MEDIUM

Short daily flashcard-style reviews (multiplication facts, spellings, sight words) using spaced repetition, separate from the weekly test.

→ Why it helps the student: Facts that need to become automatic — times tables, spellings — are exactly what spaced repetition is proven to lock in, and daily 2-minute reviews fit a child's attention span far better than weekly tests.

#	COMPONENT	WHAT HAPPENS
1	**New table**	A lightweight review_items table in PostgreSQL stores fact-level items with a next_review_date per student, using a simple SM-2-style interval algorithm.
2	**Daily surface**	The app surfaces 5-10 due items each day as a quick swipe-through card UI.
3	**Interval update**	Each answer (correct/incorrect) updates the item's next_review_date — correct answers push it further out, mistakes bring it back sooner.

3. Culturally-Localized Examples Engine

IMPACT: MEDIUM EFFORT: LOW

The tutor's examples adapt to the child's region — a Pongal-themed math problem in Tamil Nadu, a Baisakhi-themed one in Punjab — instead of one generic example set.

→ Why it helps the student: A child grasps a concept faster and remembers it longer when the example is something from their own life, not an abstract or unfamiliar scenario.

#	COMPONENT	WHAT HAPPENS
1	**Profile field**	Student profile already stores 'state' — this is read into the system prompt as a localization hint.
2	**Prompt engineering**	The fine-tuning dataset (prepare_dataset.py) is extended with regionally-varied example phrasing per state, so the model has seen this style during training, not just at inference.
3	**No new infra needed**	This is primarily a dataset and prompt change, not a new service.

4. Sibling / Near-Peer Teaching Mode

IMPACT: MEDIUM EFFORT: LOW

An older sibling (e.g. Class 5) is prompted to 'teach' a younger one (e.g. Nursery) a concept, with the AI scaffolding the explanation in the background.

→ Why it helps the student: Reflects how many rural households already function — older kids are de facto tutors — and 'learning by teaching' reinforces the older child's own understanding while genuinely helping the younger one.

#	COMPONENT	WHAT HAPPENS
1	**Family linking**	Multiple students under one parent_id can be flagged as siblings; the app suggests a paired session when both are active.
2	**Scaffolded prompt**	The LLM is fed the older child's class level and asked to coach them on how to explain the topic simply, rather than explaining it directly.
3	**Dual logging**	The session logs contributions from both children for the parent dashboard.

Tier 2 — High-Impact Builds (core learning value)

5. Misconception-Aware Mistake Diagnosis

IMPACT: HIGH EFFORT: MEDIUM

Instead of just marking an answer right or wrong, the tutor analyzes the pattern behind wrong answers (e.g. consistently miscounting, or a place-value mix-up) and targets remediation at the actual cause.

→ Why it helps the student: A child who keeps getting the same type of question wrong isn't helped by more of the same question — they're helped by someone noticing why they're wrong and fixing that specific gap.

#	COMPONENT	WHAT HAPPENS
1	**Pattern capture**	Wrong answers are logged with the student's actual working/answer, not just a correct/incorrect flag.
2	**Diagnostic prompt**	Periodically, a LangChain chain reviews a student's recent wrong answers in one subject and asks the LLM to identify a likely underlying misconception.
3	**Targeted follow-up**	The next generated exercise set is biased toward that specific misconception rather than the topic in general.

6. Reading Fluency & Pronunciation Coach

IMPACT: HIGH

EFFORT: MEDIUM

The child reads a sentence aloud; the app listens and gives gentle feedback on pronunciation and pacing — directly addressing the English-medium fluency gap this whole product exists for.

→ Why it helps the student: For a child whose home language isn't English, reading-aloud practice with real feedback is one of the highest-value things an app can offer — and nothing in the current feature set targets fluency directly.

#	COMPONENT	WHAT HAPPENS
1	**Capture**	Child reads a target sentence aloud; Whisper STT transcribes it with word-level timestamps.
2	**Comparison**	Transcribed text is diffed against the expected sentence to flag mispronounced or skipped words.
3	**Feedback**	TTS plays back the correct pronunciation of just the flagged words, followed by encouragement, not a score.

7. Frustration & Engagement Detection

IMPACT: MEDIUM-HIGH

EFFORT: MEDIUM

If a session shows signs of frustration — several wrong answers in a row, long pauses, repeated 'I don't know' — the tutor proactively switches strategy: an easier question, more encouragement, or a suggested break.

→ Why it helps the student: Children disengage quickly when stuck, and a rigid tutor that keeps pushing the same difficulty makes that worse — noticing frustration early protects both learning and the child's relationship with the app.

#	COMPONENT	WHAT HAPPENS
1	**Signal collection**	Response latency, consecutive incorrect answers, and short/uncertain replies are tracked per session in real time.
2	**Threshold check**	A simple rule layer (not a separate ML model) flags when signals cross a frustration threshold.
3	**Strategy switch**	The LangChain agent's next turn is instructed to drop difficulty, add encouragement, or suggest a short break.

8. Pre-Reader Mode for Nursery / LKG

IMPACT: HIGH for youngest band

EFFORT: MEDIUM

A voice-and-picture-only interaction mode for children too young to read any text at all — letter sounds, picture matching, no UI text dependency.

→ Why it helps the student: Nursery-age children are the most underserved part of the target range right now — a text-heavy chat UI simply doesn't work for a non-reader, no matter how good the underlying model is.

#	COMPONENT	WHAT HAPPENS
1	**UI mode switch**	Student profile's class_level=0 (Nursery) triggers a completely different React Native screen: large tappable images, no text input.
2	**Voice-first loop**	All interaction happens via TTS prompts and voice/tap responses; the LLM is constrained to very short, single-idea utterances.
3	**Visual assets**	Nano Banana Pro-generated picture sets (animals, shapes, letters) back the tap-to-respond interface.

Tier 3 — Strategic Bets (high effort, high ceiling)

9. Handwritten Worksheet Grading via Camera

IMPACT: HIGH EFFORT: HIGH

A child writes answers on paper (common where tablets aren't available); a photo of the page is checked against a generated answer key with feedback, extending the camera feature beyond printed-text translation.

→ Why it helps the student: Many rural households still rely on paper practice — this meets the student where they actually work, instead of requiring screen-only practice.

#	COMPONENT	WHAT HAPPENS
1	**Capture**	Parent/child photographs the worksheet page via the existing camera flow.
2	**Handwriting OCR**	A handwriting-capable vision model (a meaningfully harder problem than the printed-text OCR already in the app) extracts the written answers.
3	**Answer matching**	Extracted answers are compared against the question bank's stored correct_answer and explanation fields in PostgreSQL.
4	**Feedback**	Per-question feedback is generated and spoken back or shown as an overlay on the photographed page.

10. Parent Co-Learning Track

IMPACT: VERY HIGH (mission-level) EFFORT: MEDIUM-HIGH

A light, separate track that slowly builds the parent's own English vocabulary and sentence comprehension alongside what their child is learning — turning the app from a permanent translation crutch into a two-generation literacy tool.

→ Why it helps the student: This is the only feature on this list that addresses the root cause of the whole problem rather than working around it — every other feature helps the parent cope with the gap; this one actually closes it over time.

#	COMPONENT	WHAT HAPPENS
1	**New persona track**	A parent_learning_progress collection tracks vocabulary/sentence patterns the parent has been exposed to via translations.
2	**Spaced micro-lessons**	Once a week, the app surfaces 3-5 words/phrases the parent has seen repeatedly in translations, with a short audio-based mini-lesson.
3	**Tie-in to existing data**	This reuses translation history already being logged — no new data collection, just a new presentation layer on top of it.

11. WhatsApp / Voice-Call Channel

IMPACT: HIGH (reach) EFFORT: HIGH

A WhatsApp-based or basic voice-call interface for parents who aren't comfortable navigating an app UI at all, common among lower digital-literacy users on feature phones or shared devices.

→ Why it helps the student: Reaches the families this product is meant for but who would otherwise be excluded by app-literacy requirements alone — a translation or homework question becomes as simple as sending a voice note.

#	COMPONENT	WHAT HAPPENS
1	**Channel integration**	WhatsApp Business API receives a photo or voice note from the parent.
2	**Routing**	Spring Boot routes the message through the same OCR/translation or tutor pipeline already built for the app.
3	**Reply delivery**	The response (text, translated sentence, or audio) is sent back over WhatsApp, no app installation required.

12. Teacher-Linked Homework Sync

IMPACT: MEDIUM-HIGH

EFFORT: HIGH (needs school buy-in)

A child's actual school teacher can assign specific NCERT chapters or homework that syncs directly into the app's scheduling, bridging the home-school gap rather than operating purely as a separate parallel track.

→ Why it helps the student: Right now the app guesses what to review based on usage; with real teacher input, practice stays aligned with what's actually being taught in class that week.

#	COMPONENT	WHAT HAPPENS
1	**Teacher role**	Extends the existing UserRole enum (already supports TEACHER) with a lightweight web portal to assign chapters.
2	**Sync**	Assigned chapters write directly into the student's current_topics and the weekly scheduler's source data.
3	**School-level reporting**	Aggregated, anonymized progress data can optionally roll up to the teacher's view.

13. Adaptive Bandwidth-Aware Model Routing

IMPACT: MEDIUM (infra)

EFFORT: MEDIUM-HIGH

Instead of a static online/offline toggle, the app continuously measures real network quality and dynamically chooses between the full cloud model, the cached answer, or the on-device fallback model mid-session.

→ Why it helps the student: Rural connectivity often fluctuates within a single session rather than being simply 'on' or 'off' — a dynamic router keeps the experience smooth instead of the app stalling or erroring when a connection degrades.

#	COMPONENT	WHAT HAPPENS
1	**Network probe**	The React Native client periodically pings a lightweight endpoint to estimate latency/throughput.
2	**Routing decision**	A small decision layer chooses cloud LLM, topics_cache, or on-device model per-request based on the current measurement.
3	**Seamless fallback**	If a cloud call times out mid-session, the client automatically retries against the on-device model without the child noticing an error state.

Architect's Closing Note

If I had to sequence this myself: ship the Quick Wins alongside the current build, since none of them require new infrastructure and they meaningfully improve retention and learning quality almost immediately. Treat the Reading Fluency Coach and Misconception-Aware Diagnosis as the next real milestone after the MVP is stable — they're the two features most directly tied to why this product needs to exist in the first place. The Parent Co-Learning Track is the one I'd protect from being deprioritized indefinitely: every other feature works around the parent's English gap; this is the only one that closes it.